Practice Set: new and delete Operators in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Differentiate between **Static/Automatic Memory** allocation on the Call Stack and **Dynamic Memory** allocation on the Heap (Free Store) in C++. Why do real-world applications require dynamic memory management?

Question 2: Define the new and delete operators in C++. Explain the step-by-step memory changes occurring in RAM when the statement int* ptr = new int; *ptr = 42; is executed.

Question 3: Detail the single variable allocation and deallocation syntax using new and delete. Why is setting a pointer to nullptr immediately after calling delete considered a standard best practice?

Question 4: What two steps occur automatically when a class object is dynamically created using new ClassName(arguments) on the Heap? How does calling delete on that object pointer complete its lifecycle?

Question 5: State the mandatory pairing rule between array allocation and deallocation syntax (new[] vs delete[]). What architectural corruption or undefined behaviors occur if a programmer uses single delete on an array pointer?

Question 6: Compare C-style malloc() / free() with modern C++ new / delete across nature, constructor/destructor execution, type safety, size calculation, and failure handling.

Question 7: Define a **Memory Leak**. Explain how a memory leak develops in a C++ program and why it poses severe performance issues for long-running software processes.

Question 8: Define a **Dangling Pointer** and a **Double Free Error**. Explain what causes each runtime defect and why setting pointers to nullptr prevents both issues.

Question 9: Write a complete C++ program demonstrating dynamic allocation and initialization of a single primitive variable (int). Show pointer dereferencing to update its value, printing its memory address and value, and releasing the memory cleanly.

Question 10: Write a complete C++ program that dynamically allocates a Student object on the Heap with parameterized values (name and age). Invoke a member function using the arrow (->) operator, and properly deallocate the heap memory to execute its destructor.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): In which memory region (Stack or Heap) is dynamic memory allocated when invoking new?

Question 12 (Viva): Does the new operator automatically call object constructors when instantiating custom class types on the Heap?

Question 13 (Viva): What operator must be used to deallocate memory allocated with int* arr = new int[50]?

Question 14 (Viva): What exception does the default new operator throw when the operating system fails to allocate the requested dynamic memory?

Question 15 (Viva): Why is calling malloc() insufficient for creating C++ class objects dynamically on the Heap?

Question 16 (Viva): What happens to the local pointer variable located on the Call Stack when delete ptr; is executed?

Question 17 (Viva): Is calling delete on a pointer holding nullptr safe or dangerous in C++?

Question 18 (Viva): What syntax operator is used to access class members through an object pointer allocated on the Heap?

Question 19 (Viva): What major C++ feature introduced in C++11 provides automatic resource management (RAII) to replace manual new and delete calls?

Question 20 (Viva): What is the default return type of C-style malloc(), and how does it differ from the return type of new in C++?

Answers & Explanations

Answer: 01

Stack vs. Heap Memory:

Static/Automatic Memory (Stack): Allocated automatically on the Call Stack for local variables. Memory size must be known at compile time, and storage is automatically reclaimed when the function scope terminates.

Dynamic Memory (Heap): Allocated at runtime on the Heap (Free Store). Storage remains allocated until explicitly freed by the programmer, persisting across function calls.

Necessity: Real-world applications require dynamic memory when the exact data size depends on user input at runtime (e.g., reading dynamic database records) or when object lifetimes need to span across multiple function scopes.

Answer: 02

new Operator: Computes the required byte size, allocates contiguous memory on the Heap at runtime, invokes constructors for class objects, and returns a typed memory pointer.

delete Operator: Releases dynamically allocated Heap memory back to the Operating System and invokes object destructors prior to reclaiming memory.

Execution Mechanics for int* ptr = new int; *ptr = 42;:

A pointer variable ptr (e.g., 8 bytes on a 64-bit architecture) is created on the Call Stack.

The new operator requests 4 bytes of contiguous dynamic memory from the Heap.

The Heap memory address (e.g., 0x00508000) is returned and stored inside ptr on the stack.

The dereference assignment *ptr = 42; writes the integer value 42 directly into that Heap memory address.

Answer: 03

Syntax:

```
int* ptr = new int;
int* ptr2 = new int(100);
delete ptr;
ptr = nullptr;
```

Nullifying Best Practice: Calling delete ptr; frees the dynamic Heap block but leaves ptr containing the old memory address (a Dangling Pointer). Resetting ptr = nullptr; ensures subsequent safety checks prevent dereferencing deallocated memory.

Answer: 04

Dynamic Object Allocation Steps:

Memory Allocation: Memory space is allocated on the Heap sufficient to store all data attributes of the class.

Constructor Invocation: The specified constructor ClassName(arguments) executes immediately to initialize those attributes.

Lifecycle Completion via delete: Executing delete objectPtr; first invokes the class destructor to clean up internal object resources, and then reclaims the allocated Heap memory block for the operating system.

Answer: 05

Mandatory Rule: Memory allocated using array syntax new Type[size] must strictly be released using array deallocation syntax delete[] arrPtr;.

Consequences of Mismatch: Calling scalar delete ptr; on an array pointer invokes the destructor for only the first element, leaving remaining array elements un-destructed and corrupting Heap allocator metadata, which causes runtime segmentation faults or undefined behavior.

Answer: 06

Feature	C-Style (malloc / free)	Modern C++ (new / delete)
Nature	Standard library functions (<cstdlib>)	Built-in C++ language operators
Constructors / Destructors	Does not execute constructors or destructors	Automatically calls constructors on new and destructors on delete
Type Safety & Casting	Returns void*; requires explicit type casting	Returns exact typed pointer (e.g., int*); strongly type-safe
Size Calculation	Manual byte calculation (sizeof(type) * count)	Automatic byte calculation by compiler
Failure Handling	Returns NULL / nullptr on memory failure	Throws std::bad_alloc exception (or returns nullptr with nothrow)
Array Syntax	Same free(ptr) function for single variables and arrays	Requires distinct delete[] syntax for array cleanup

Answer: 07

Definition: A Memory Leak occurs when memory is allocated on the Heap via new, but the program loses all reference pointers to that block without invoking delete.

System Impact: The memory remains locked and unusable by other processes. In long-running applications (like web servers or games), continuous memory leaks progressively consume system resources until host memory is exhausted, causing application slowdowns or crashes.

Answer: 08

Dangling Pointer: Occurs when delete ptr; is called to free dynamic memory, but ptr is not updated. The pointer still holds the deallocated RAM address, and dereferencing it causes memory corruption or crashes.

Double Free Error: Occurs when attempting to call delete ptr; twice on the exact same active memory address. This corrupts internal Heap allocator data structures, causing program termination.

Prevention via nullptr: Setting ptr = nullptr; after deletion fixes both issues because dereferencing null can be caught safely, and deleting a null pointer (delete nullptr;) is explicitly guaranteed to be a safe no-op in C++.

Answer: 09

```
#include <iostream>
using namespace std;
int main()
{
   int* ptr = new int;
   *ptr = 42;

   cout << "Allocated Heap Address: " << ptr << endl;
   cout << "Value at Address (*ptr): " << *ptr << endl;

   delete ptr;
   ptr = nullptr;

   return 0;
}
```

Answer: 10

```
#include <iostream>
#include <string>
using namespace std;
class Student
{
  public:
   string name;
   int age;

   Student(string n, int a)
   {
      name = n;
      age = a;
      cout << "Constructor: Allocated and initialized Student object." << endl;
   }
```

```
    // Destructor
    ~Student()
    {
        cout << "Destructor: Student object cleaned up." << endl;
    }

    void display()
    {
        cout << "Name: " << name << " | Age: " << age << endl;
    }
};
int main()
{
    Student* s1 = new Student("Alice", 20);

    // Access member methods via arrow (->) operator
    s1->display();

    // Deallocate memory, triggering ~Student() destructor
    delete s1;
    s1 = nullptr;

    return 0;
}
```

Answer: 11
The **Heap** (also known as the Free Store).

Answer: 12
Yes, new automatically allocates memory and executes the corresponding constructor.

Answer: 13
The **delete[]** array deallocation operator (delete[] arr;).

Answer: 14
It throws a **std::bad_alloc** exception.

Answer: 15
Because malloc() allocates raw bytes without invoking object constructors.

Answer: 16
The stack pointer variable remains allocated on the Call Stack holding the freed address until its scope exits, which is why it should be set to nullptr.

Answer: 17
It is **completely safe**; deleting nullptr is guaranteed to perform no operation in C++.

Answer: 18
The **Arrow operator (->)**.

Answer: 19
Smart Pointers (std::unique_ptr, std::shared_ptr).

Answer: 20
malloc() returns a generic void* pointer requiring explicit type casting, whereas new returns a **typed pointer** matching the allocated data type.